

Hash Functions

Exam Notes for Exercises 11.3-1 to 11.3-6

Main Ideas

What to Remember

- When keys are long strings, compare hash values first and compare strings only when hash values match.
- To compute a large radix number modulo m , use Horner's rule and reduce modulo m at every step.
- If the radix is congruent to 1 modulo m , then the hash depends only on the sum of the character values, so permutations collide.
- Universal hashing bounds the collision probability over a random choice of hash function.

1 Exercise 11.3-1

Question

Suppose we search a linked list of length n . Each element stores a long string key k and also its hash value $h(k)$. How can we use the hash values when searching for a given key?

Answer

Compute the hash value of the search key once. Then, while scanning the list, compare the stored hash value before comparing the full string.

Method

Let the search key be q .

1. Compute $h(q)$ once.
2. For each node with stored key k and stored hash value $h(k)$:
 - if $h(k) \neq h(q)$, skip the expensive string comparison;
 - if $h(k) = h(q)$, then compare the actual strings k and q .

This is correct because if $k = q$, then necessarily $h(k) = h(q)$. If the hash values differ, the keys cannot be equal. If the hash values match, the keys might still be different because of a collision, so we must compare the strings.

Write This in Exam

First compute the hash of the search key. In each node, compare hash values first. Only if the hash values agree do we compare the long strings. This avoids most expensive string comparisons, while still being correct because collisions are checked by comparing the actual keys.

2 Exercise 11.3-2

Question

A string of r characters is treated as a radix-128 number and hashed by the division method modulo m . The full number may require many words. How can we compute the hash value using only constant extra storage?

Answer

Use Horner's rule and reduce modulo m after processing each character.

Computation

Let the character values be

$$c_1, c_2, \dots, c_r, \quad 0 \leq c_i < 128.$$

The string represents the number

$$K = c_1 128^{r-1} + c_2 128^{r-2} + \dots + c_{r-1} 128 + c_r.$$

We want

$$h(K) = K \bmod m.$$

Compute:

$$x_0 = 0,$$

and for $i = 1, 2, \dots, r$,

$$x_i = (128x_{i-1} + c_i) \bmod m.$$

Then output

$$x_r.$$

At every step, $0 \leq x_i < m$, so we store only the current accumulator, the current character, and m . Thus the algorithm uses only a constant number of machine words outside the string itself.

Write This in Exam

Scan the string from left to right and maintain one accumulator:

$$x \leftarrow 0, \quad x \leftarrow (128x + c_i) \bmod m$$

for each character c_i . At the end, $x = K \bmod m$. Since $x < m$ always, only constant extra storage is needed.

Common Mistake

Do not just add the character values modulo m . The radix-128 positions matter. The correct method is

$$x \leftarrow (128x + c_i) \bmod m.$$

3 Exercise 11.3-3

Question

Let

$$h(k) = k \bmod m, \quad m = 2^p - 1,$$

where a character string is interpreted in radix 2^p . Show that permuting the characters of a string does not change its hash value. Give an application where this is undesirable.

Answer

Since the radix is 2^p and

$$2^p \equiv 1 \pmod{2^p - 1},$$

each power of the radix is congruent to 1 modulo m . Therefore the hash depends only on the sum of the character values, not their order.

Proof

Let the radix be

$$b = 2^p.$$

Then

$$m = b - 1,$$

so

$$b \equiv 1 \pmod{m}.$$

Hence, for every $j \geq 0$,

$$b^j \equiv 1 \pmod{m}.$$

If the string has character values

$$a_0, a_1, \dots, a_{r-1},$$

then its radix- b value is

$$K = a_0 b^{r-1} + a_1 b^{r-2} + \dots + a_{r-2} b + a_{r-1}.$$

Modulo m ,

$$K \equiv a_0 + a_1 + \dots + a_{r-1} \pmod{m}.$$

Thus the hash value depends only on the sum of the character values. Any permutation of the same characters has the same sum, so it has the same hash value.

Undesirable Application

This is bad for hashing words or identifiers in a compiler symbol table, dictionary, search engine, or password system. For example, anagrams such as “post”, “pots”, and “stop” would always collide, even though they are different strings.

Write This in Exam

Let $b = 2^p$ and $m = b - 1$. Since $b \equiv 1 \pmod{m}$, every power b^j is also congruent to 1 modulo m . Therefore a string value

$$\sum_j a_j b^j$$

hashes to

$$\sum_j a_j \pmod{m}.$$

This sum is unchanged by permuting characters, so all permutations collide. This is undesirable when hashing strings where order matters, such as program identifiers or dictionary words.

4 Exercise 11.3-4

Question

For a hash table of size

$$m = 1000$$

and multiplication-method hash function

$$h(k) = \lfloor m(kA \bmod 1) \rfloor, \quad A = \frac{\sqrt{5} - 1}{2},$$

compute the locations for keys 61, 62, 63, 64, 65.

Answer

Using

$$A = \frac{\sqrt{5} - 1}{2} \approx 0.6180339887,$$

we compute the fractional part of kA and multiply it by 1000.

k	kA	$kA \bmod 1$	$h(k)$
61	37.7000733	0.7000733	700
62	38.3181073	0.3181073	318
63	38.9361413	0.9361413	936
64	39.5541753	0.5541753	554
65	40.1722093	0.1722093	172

Write This in Exam

$$h(61) = 700, \quad h(62) = 318, \quad h(63) = 936, \quad h(64) = 554, \quad h(65) = 172.$$

5 Exercise 11.3-5

Question

Let H be a family of hash functions from a finite set U to a finite set B . Suppose H is ε -universal, meaning that for all distinct $k, l \in U$,

$$\Pr_{h \in H} \{h(k) = h(l)\} \leq \varepsilon.$$

Show that

$$\varepsilon \geq \frac{1}{|B|} - \frac{1}{|U|}.$$

Answer

Count colliding pairs in two ways. Any single hash function must create many collisions on average by the pigeonhole principle, while ε -universality upper-bounds the total number of collisions.

Proof

Let

$$u = |U|, \quad b = |B|, \quad N = |H|.$$

For a fixed hash function h , let

$$n_y = |\{x \in U \mid h(x) = y\}|$$

for each $y \in B$. The number of unordered colliding pairs under h is

$$C_h = \sum_{y \in B} \binom{n_y}{2} = \frac{1}{2} \left(\sum_{y \in B} n_y^2 - u \right).$$

Since

$$\sum_{y \in B} n_y = u,$$

Cauchy's inequality gives

$$\sum_{y \in B} n_y^2 \geq \frac{u^2}{b}.$$

Therefore

$$C_h \geq \frac{1}{2} \left(\frac{u^2}{b} - u \right).$$

Summing over all N hash functions gives at least

$$N \cdot \frac{1}{2} \left(\frac{u^2}{b} - u \right)$$

colliding pairs in total.

On the other hand, ε -universality says that each unordered pair $\{k, l\}$ collides for at most εN hash functions. There are

$$\binom{u}{2} = \frac{u(u-1)}{2}$$

unordered pairs. Thus the total number of collisions is at most

$$\varepsilon N \frac{u(u-1)}{2}.$$

Hence

$$N \cdot \frac{1}{2} \left(\frac{u^2}{b} - u \right) \leq \varepsilon N \frac{u(u-1)}{2}.$$

Canceling $N/2$ gives

$$\frac{u^2}{b} - u \leq \varepsilon u(u-1).$$

Thus

$$\varepsilon \geq \frac{u-b}{b(u-1)}.$$

Since

$$\frac{u-b}{b(u-1)} \geq \frac{u-b}{bu} = \frac{1}{b} - \frac{1}{u},$$

we get

$$\varepsilon \geq \frac{1}{|B|} - \frac{1}{|U|}.$$

Write This in Exam

Let $u = |U|$ and $b = |B|$. For any fixed h , if n_y keys hash to y , then the number of colliding pairs is

$$\sum_y \binom{n_y}{2} = \frac{1}{2} \left(\sum_y n_y^2 - u \right) \geq \frac{1}{2} \left(\frac{u^2}{b} - u \right).$$

But ε -universality implies the total number of colliding pairs over all functions is at most

$$\varepsilon |H| \binom{u}{2}.$$

Comparing the lower and upper bounds yields

$$\varepsilon \geq \frac{u-b}{b(u-1)} \geq \frac{1}{b} - \frac{1}{u}.$$

6 Exercise 11.3-6

Question

Let U be the set of n -tuples over $\mathbb{Z}_p$, and let $B = \mathbb{Z}_p$, where p is prime. For $b \in \mathbb{Z}_p$, define

$$h_b(\langle a_0, a_1, \dots, a_{n-1} \rangle) = \left(\sum_{j=0}^{n-1} a_j b^j \right) \bmod p.$$

Let

$$H = \{h_b \mid b \in \mathbb{Z}_p\}.$$

Show that H is $((n-1)/p)$ -universal.

Answer

For two distinct tuples, a collision occurs only when a nonzero polynomial of degree at most $n-1$ evaluates to 0 over the field $\mathbb{Z}_p$. Such a polynomial has at most $n-1$ roots.

Proof

Take two distinct tuples

$$a = \langle a_0, a_1, \dots, a_{n-1} \rangle$$

and

$$c = \langle c_0, c_1, \dots, c_{n-1} \rangle.$$

They collide under h_b exactly when

$$h_b(a) = h_b(c).$$

That is,

$$\sum_{j=0}^{n-1} a_j b^j \equiv \sum_{j=0}^{n-1} c_j b^j \pmod{p}.$$

Equivalently,

$$\sum_{j=0}^{n-1} (a_j - c_j) b^j \equiv 0 \pmod{p}.$$

Define the polynomial

$$P(x) = \sum_{j=0}^{n-1} (a_j - c_j) x^j.$$

Since $a \neq c$, not all coefficients are zero. Therefore $P(x)$ is a nonzero polynomial over the field $\mathbb{Z}_p$, with degree at most $n - 1$.

A nonzero polynomial of degree at most $n - 1$ over a field has at most $n - 1$ roots. Therefore there are at most $n - 1$ values of $b \in \mathbb{Z}_p$ for which

$$h_b(a) = h_b(c).$$

Since b is chosen uniformly from p possible values,

$$\Pr_{b \in \mathbb{Z}_p} \{h_b(a) = h_b(c)\} \leq \frac{n-1}{p}.$$

Thus H is

$$\frac{n-1}{p}\text{-universal}.$$

Write This in Exam

For distinct tuples a and c , a collision means

$$\sum_{j=0}^{n-1} (a_j - c_j) b^j \equiv 0 \pmod{p}.$$

This is a nonzero polynomial in b of degree at most $n - 1$. Over the field $\mathbb{Z}_p$, it has at most $n - 1$ roots. Since there are p possible choices of b , the collision probability is at most

$$\frac{n-1}{p}.$$

Therefore the family is $((n-1)/p)$ -universal.

Final Summary

One-Line Answers

Exercise	Key answer
11.3-1	Compare hash values first; compare long strings only if hashes match.
11.3-2	Use Horner's rule: $x \leftarrow (128x + c_i) \bmod m$.
11.3-3	Since $2^p \equiv 1 \pmod{2^p - 1}$, the hash depends only on the sum of characters, so permutations collide.
11.3-4	The keys 61, 62, 63, 64, 65 map to 700, 318, 936, 554, 172.
11.3-5	Count colliding pairs; every ε -universal family needs $\varepsilon \geq 1/ B - 1/ U $.
11.3-6	A collision is a root of a nonzero degree- $(n - 1)$ polynomial over $\mathbb{Z}_p$, so probability is at most $(n - 1)/p$.